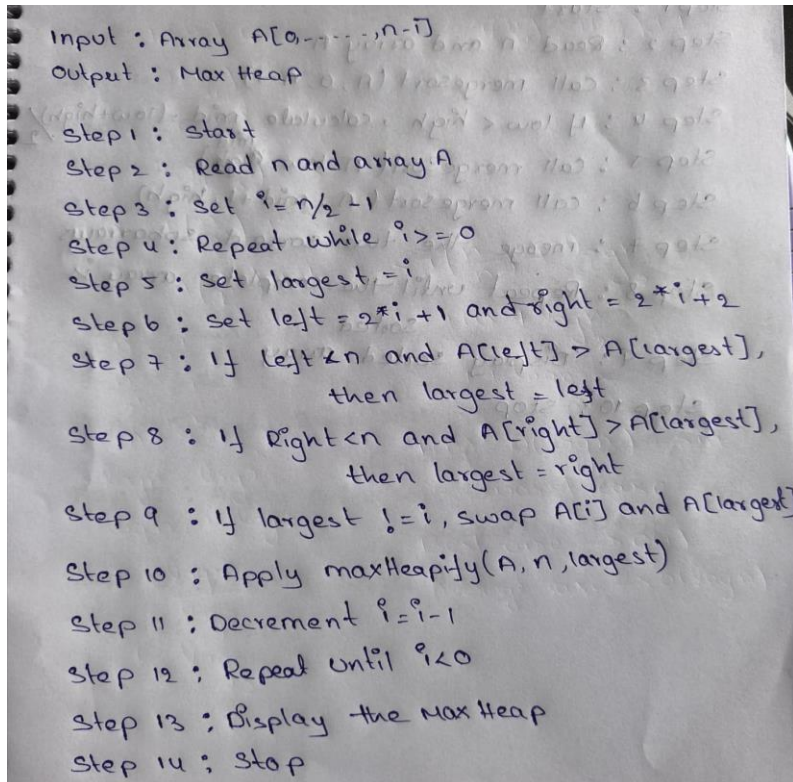


Practical - 3

Aim: Implementation of max-heap sort algorithm

Max-heap sort

Algorithm:



Input : Array $A[0 \dots n-1]$
Output : Max Heap
Step 1 : Start
Step 2 : Read n and array A
Step 3 : Set $i = n/2 - 1$
Step 4 : Repeat while $i \geq 0$
Step 5 : Set $\text{largest} = i$
Step 6 : Set $\text{left} = 2*i + 1$ and $\text{right} = 2*i + 2$
Step 7 : If $\text{left} < n$ and $A[\text{left}] > A[\text{largest}]$,
then $\text{largest} = \text{left}$
Step 8 : If $\text{right} < n$ and $A[\text{right}] > A[\text{largest}]$,
then $\text{largest} = \text{right}$
Step 9 : If $\text{largest} \neq i$, swap $A[i]$ and $A[\text{largest}]$
Step 10 : Apply $\text{maxHeapify}(A, n, \text{largest})$
Step 11 : Decrement $i = i - 1$
Step 12 : Repeat until $i < 0$
Step 13 : Display the Max Heap
Step 14 : Stop

Program:

```
#include <iostream>
using namespace std;
class MaxHeap
{
public:
    int arr[100], n;
    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;
        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++)
        {
            cin >> arr[i];
        }
    }
}
```

```
void printArray()
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void maxheapify(int arr[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
    {
        largest = left;
    }

    if (right < n && arr[right] > arr[largest])
    {
        largest = right;
    }

    if (largest != i)
    {
        swap(arr[i], arr[largest]);
        maxheapify(arr, n, largest);
    }
}

void heapsort()
{
    for (int i = n / 2 - 1; i >= 0; i--)
    {
        maxheapify(arr, n, i);
    }
    for (int i = n - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);
        maxheapify(arr, i, 0);
    }
}

};

int main()
{
    MaxHeap mh;
```

```
mh.arrayInput();

cout << "\nBefore Sorting: ";
mh.printArray();

mh.heapsort();

cout << "After Sorting: ";
mh.printArray();

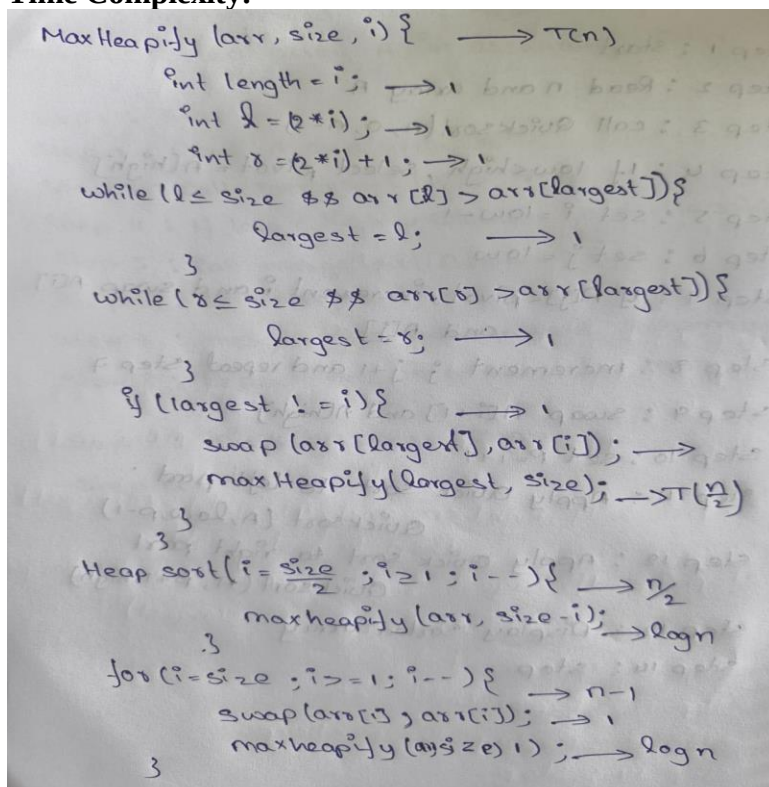
return 0;
}
```

Output:

```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\heapify.exe"
Enter the elements: 12
5
3
26
38
9

Before Sorting: 12 5 3 26 38 9
After Sorting: 3 5 9 12 26 38
```

Time Complexity:



MaxHeapify (arr, size, i) { $\rightarrow T(n)$

 int length = i; $\rightarrow 1$

 int l = 2*i; $\rightarrow 1$

 int r = 2*i + 1; $\rightarrow 1$

 while (l ≤ size && arr[l] > arr[largest]) {

 largest = l; $\rightarrow 1$

 }

 while (r ≤ size && arr[r] > arr[largest]) {

 largest = r; $\rightarrow 1$

 }

 if (largest != i) {

 swap (arr[largest], arr[i]); $\rightarrow 1$

 maxHeapify (largest, size); $\rightarrow T(\frac{n}{2})$

 }

Heap sort (i = size/2; i ≥ 1; i--) { $\rightarrow \frac{n}{2}$

 maxHeapify (arr, size-i); $\rightarrow \log n$

 for (i = size; i > 1; i--) { $\rightarrow n-1$

 swap (arr[1], arr[i]); $\rightarrow 1$

 maxHeapify (arr, size-i); $\rightarrow \log n$

 }

For maxHeapify

$$T(n) = T\left(\frac{n}{2}\right) + 1$$

using master method

$$f(n) = 1$$

$$a=1, b=2, k=0, p=0$$

$$\log_a n = \log_2 n = \frac{\log n}{\log 2} = 0 = k$$

$$p > -1 \Rightarrow 0 > -1$$

$$T(n) = O(n^k \log^{p+1} n)$$

$$= O(n^0 \log^{0+1} n)$$

$$= O(1 \log n)$$

maxHeapify = $O(\log n)$

Build max heap

$$T(n) = \frac{n}{2} \times \log n$$

$$= O(n \log n)$$

Heap sort

$$T(n) = (n-1) \times \log n$$

$$= O(n \log n)$$

Overall Heap sort

$$T(n) = O(n \log n) + O(n \log n)$$

$$= O(n \log n) \rightarrow \text{Best case}$$

in all cases

- ① Best case = $O(n \log n)$
- ② Average case = $O(n \log n)$
- ③ Worst case = $O(n \log n)$

worst case:

$$T(n) = T\left(\frac{n}{2}\right) + O(1)$$

$$= O(\log n)$$

for loop $n-1$ times

$$T(n) = O(n) + (n-1) \times O(\log n)$$

$$= O(n \log n)$$

Average case:-
Same like the worst case.

Conclusion:

The Max-Heap Sort algorithm was successfully implemented using the Max-Heapify method. The largest element is repeatedly placed at the end of the array, resulting in a sorted array in ascending order. The time complexity of Heap Sort is $O(n \log n)$.